Обзор статьи

Characterization of Large Language Model Development in the Datacenter

Характеризация разработки больших языковых моделей в дата-центре

Студенты АлтГТУ, группа 8 ПИ-51
Бельш А.Е.
Воробьев К.В.
Мэн Ч.
Ошурков Н.А.

2026 г.

Анализ рабочих нагрузок LLM в дата-центре Acme

Крупные языковые модели достигли впечатляющих результатов, но их разработка на больших GPU-кластерах сопряжена с особыми проблемами: частые сбои, сложные стратегии параллелизации и несбалансированное использование ресурсов.
В этой работе представлено шестимесячное эмпирическое исследование трассировок из Acme (кластеры Seren и Kalos, всего 4704 GPU A100) и разработаны системные решения для повышения надежности и эффективности.

Контекст и мотивация

Большие языковые модели (LLM) требуют миллиардов параметров и огромных объёмов данных, поэтому компании создают крупные GPU-кластеры для их обучения.
Однако многие выводы из прежних анализов глубокого обучения (DL) неприменимы к LLM из-за:

- новой парадигмы: масштабное самоконтролируемое предварительное обучение (self-supervised pretraining);
- унифицированной архитектуры: все LLM используют Transformers;
- специализированного программного стека: Deepspeed, Megatron, Alpa и др.

Методология: трассировки Acme

Анализ основан на данных за 6 месяцев (март–август 2023) из двух кластеров для LLM:

- **Seren:** 2288 GPU A100;
- **Kalos:** 2416 GPU A100.

Источники данных:

- журналы планировщика задач;
- мониторинг инфраструктуры (Prometheus, NVIDIA DCGM);
- журналы выполнения и профилирование.

Трассировки охватывают сотни тысяч заданий CPU/GPU и детальные метрики оборудования.

Ключевые наблюдения — время выполнения и задержки

- **Короткая продолжительность заданий:** Средняя длительность заданий LLM в 2,7–12,8 раз короче, чем в ранее опубликованных DL-трассировках. Это связано с множеством краткосрочных задач, таких как оценка и тюнинг.
- **Несправедливая задержка в очереди:** Короткие оценочные задания испытывают наибольшие задержки в очереди, потому что ресурсы преимущественно резервируются для долгих предобучающих задач (pretraining).

Ключевые наблюдения — использование ресурсов

Несбалансированное потребление:

- Предобучение (pretraining) — 3.2% заданий, но потребляет 94.0% GPU-времени (Kalos);
- Оценка (evaluation) — 92.9% заданий, но использует только 0.8% ресурсов.

Недоиспользование сопутствующих ресурсов:

- CPU, хост-память и сеть часто не загружены;
- GPU и память GPU показывают высокие медианные значения:
  - Kalos: GPU-память медиана 75% (60 ГБ), GPU-использование 99%

Проблемы в оценочных заданиях

Профилирование показало значительное время ожидания и простоя GPU в оценочных конвейерах:

- Пример: HumanEval тратит 29.5% времени на загрузку модели и предобработку данных;
- 19.0% времени уходит на проверку корректности синтезированных программ;
- Лишь около 50% времени используется непосредственно для вывода на GPU.

Следствие: это удлиняет очереди при массовой оценке и увеличивает задержки выполнения заданий.

Системные решения Acme — обзор

На основе наблюдений разработаны две ключевые системы:

1. **Отказоустойчивое предварительное обучение:** для диагностики сбоев и автоматического восстановления.
2. **Раздельное планирование оценки:** для ускорения обратной связи и сокращения простоя GPU.

Отказоустойчивое предварительное обучение

- Частые асинхронные контрольные точки (checkpoints) для быстрого восстановления.
- Диагностика корневых причин с помощью эвристик и LLM-помощника.
- Инструменты для обнаружения неисправных узлов и автоматического перезапуска из корректной контрольной точки.

Эффект: ускорение проверки восстановлений в 3.6–58.7 раза и значительное сокращение ручного вмешательства.

Раздельное планирование оценки и выводы

Развязка извлечения модели:

- Избегает конфликтов при удалённой загрузке модели за счёт отдельной фазы извлечения.

Развязка вычисления метрик:

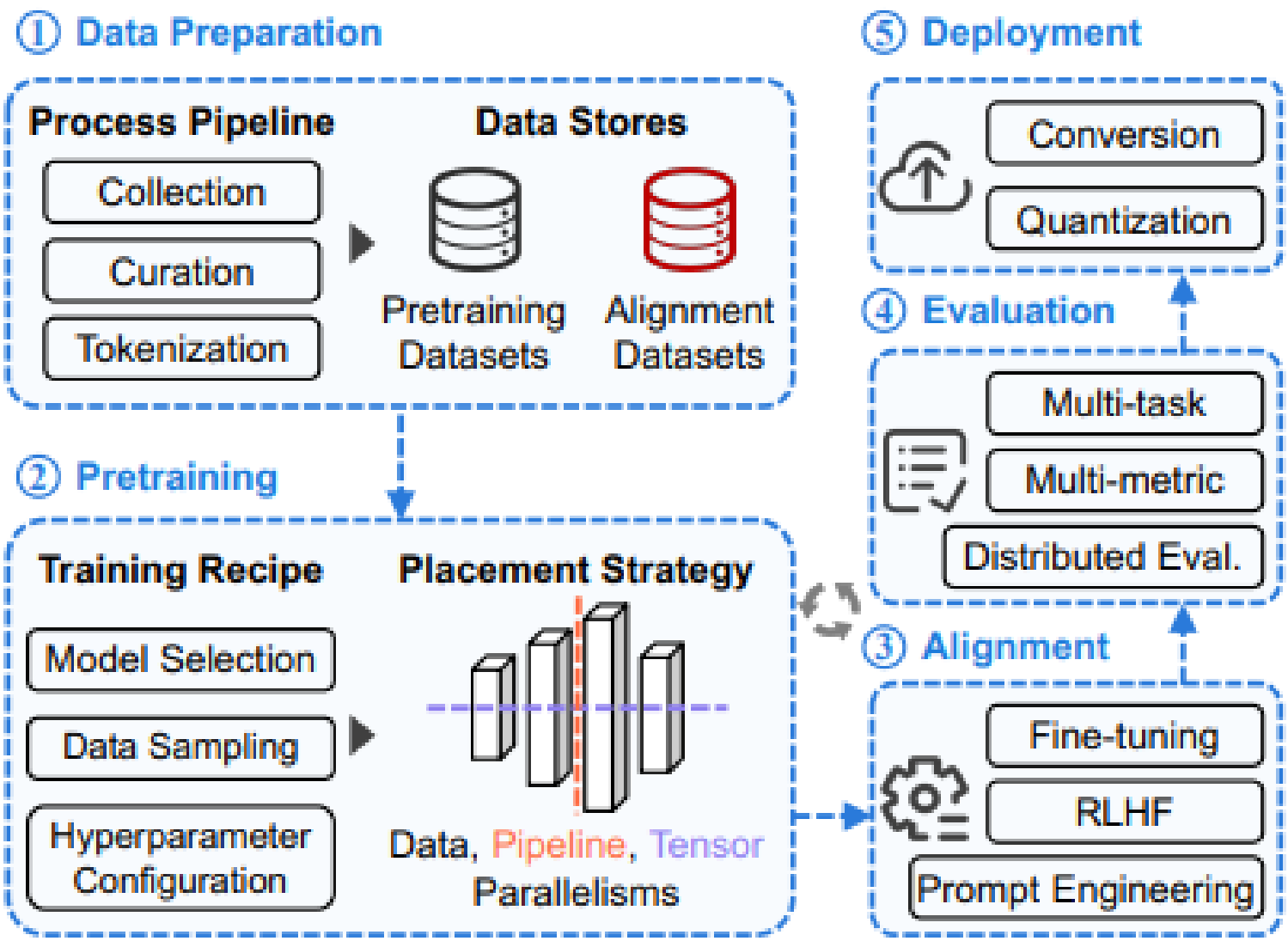
- Минимизирует простой GPU, параллелит вычисление метрик и балансирует нагрузку по предзнаниям и наборам данных.

Практический эффект:

- Сокращение времени оценки до 1.8x.
- Повышена своевременная обратная связь по качеству модели.

Ресурсы: трассировки и код опубликованы на `https://github.com/InternLM/AcmeTrace`.

Конвейер разработки LLM

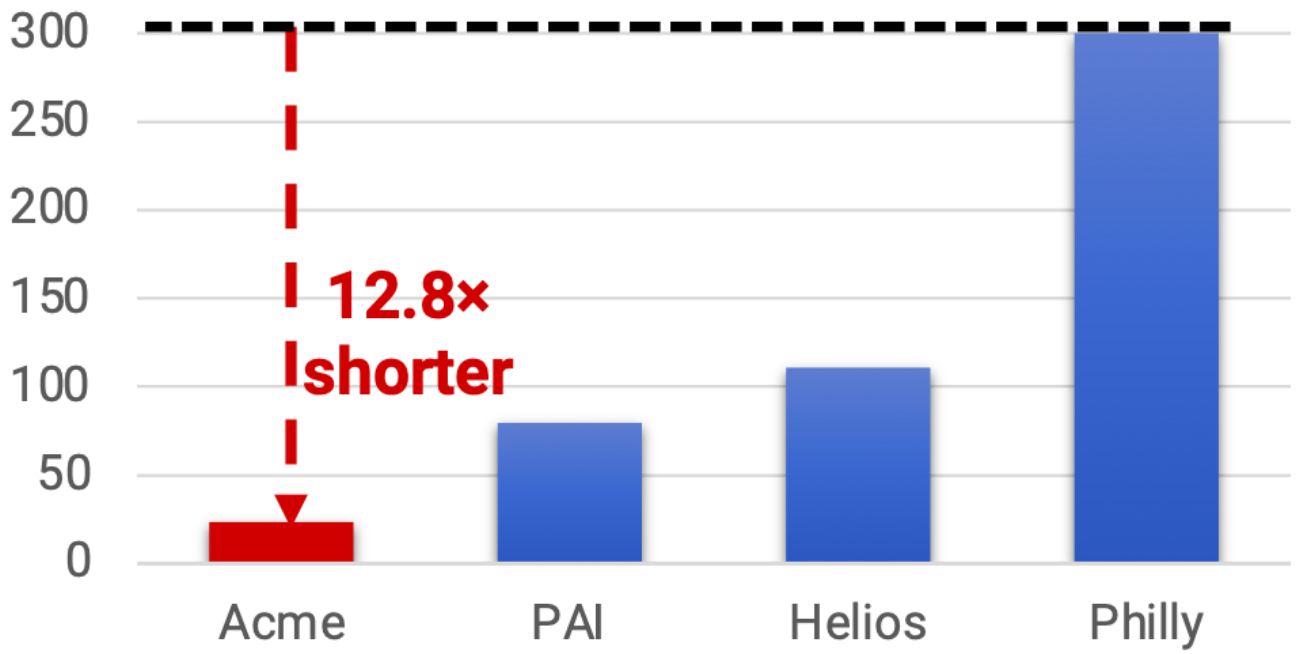


Длительность выполнения задач

Средняя длительность выполнения задач в **2.7–12.8×** *короче*, чем в других DL-трейсах.

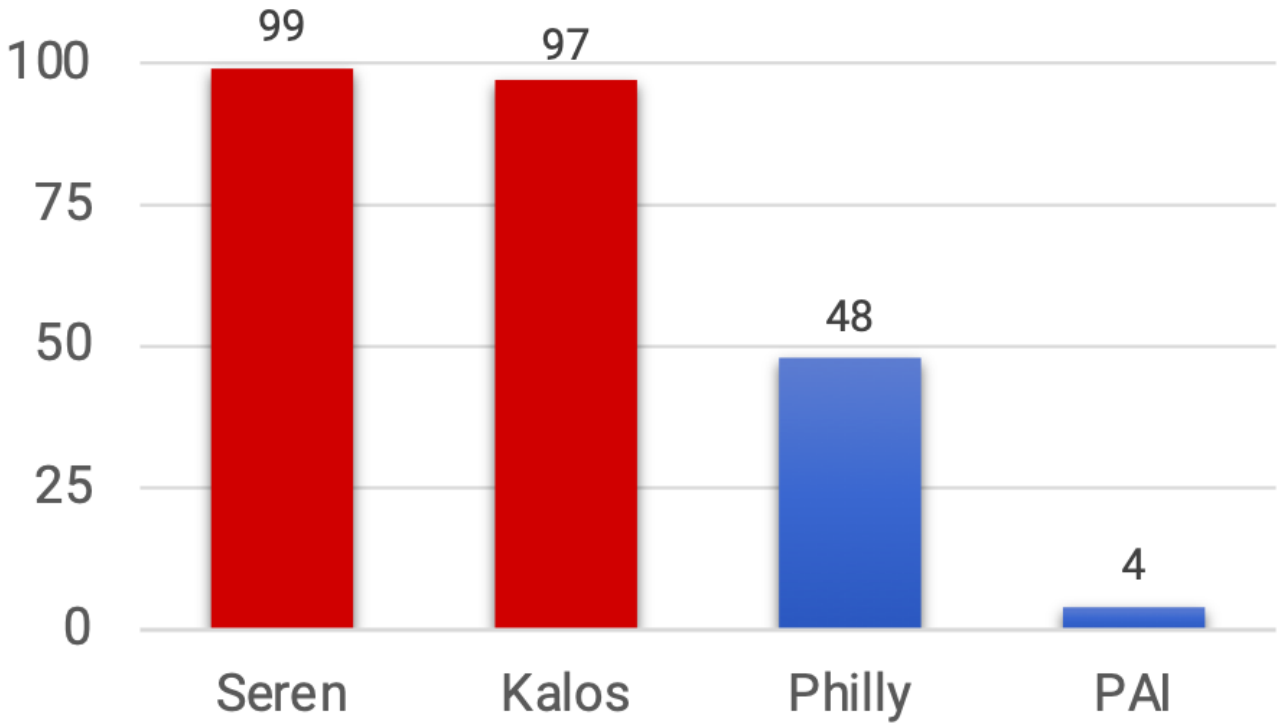
Причины:

- Более продвинутые мощности.
- Много небольших задач для GPU.
- Одна задача использует всего от 5-20 GPU.
- Много задач, которые нельзя выполнить.



Нагрузка на GPU

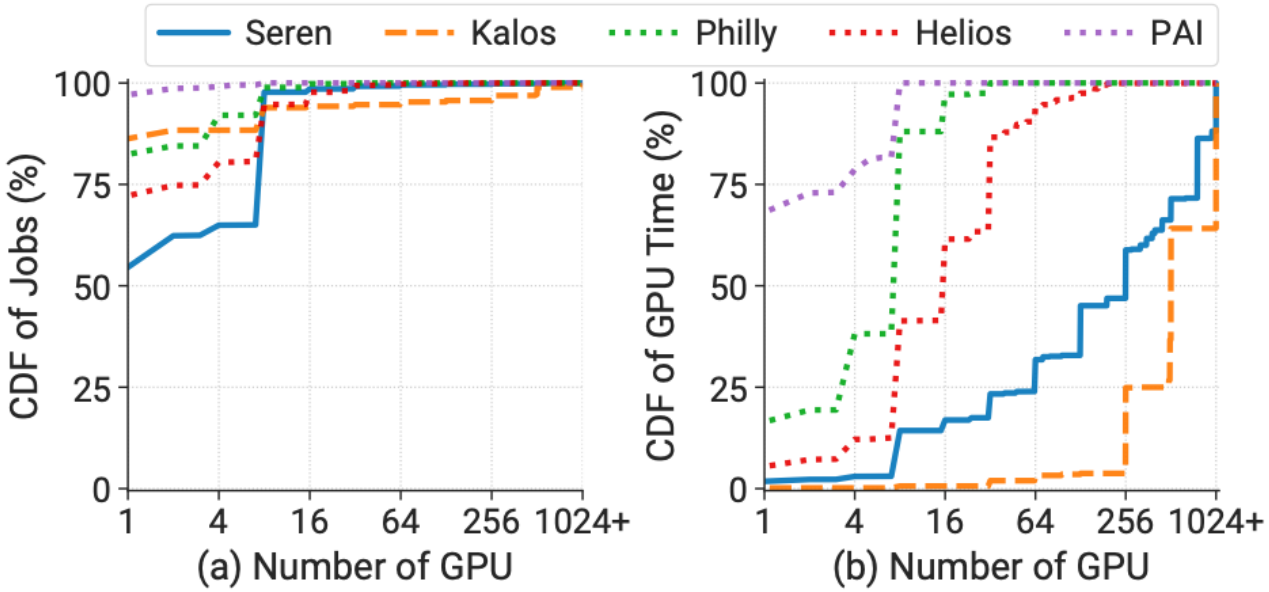
- LLM - вычислительно зависимые системы.
- Много задач не доходит до GPU, отсеиваясь еще на этапе инициализации.



среднее использование GPU

Распределение нагрузки

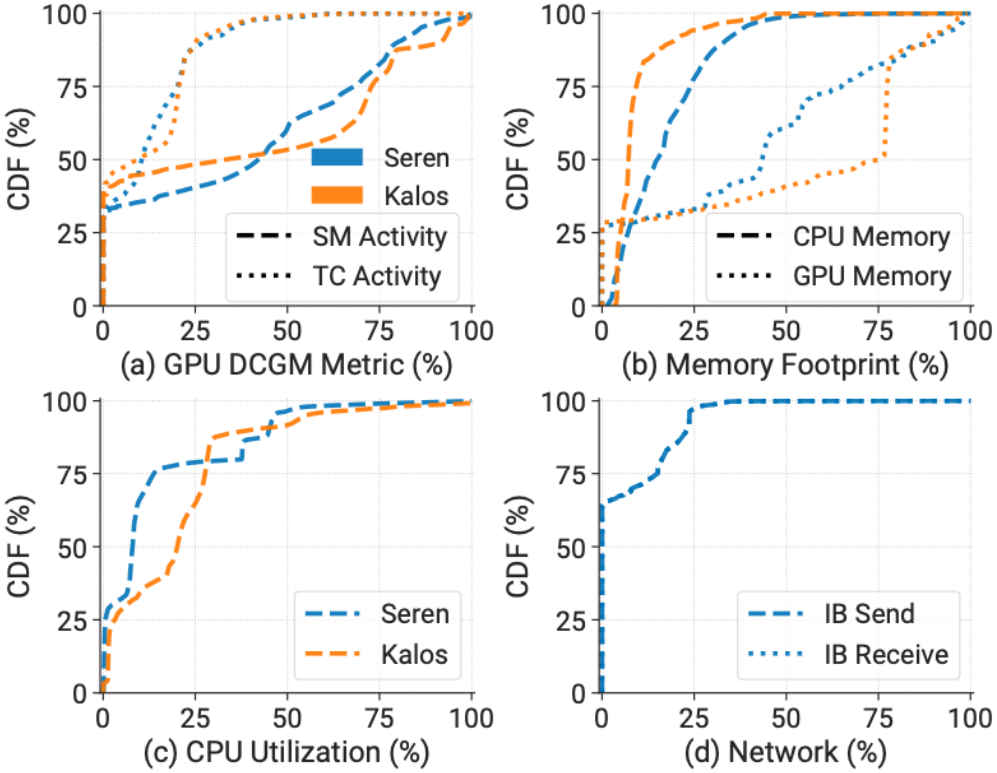
- Половина задач может выполнятся только на 1 GPU.
- Большую часть времени занимают задачи, где требуется много GPU.



распределение нагрузки в зависимости от количества запрошенных GPU

Нагрузка на инфраструктуру

- Большое потребление памяти GPU, уже при около 50% нагрузке на кластере Kalos.
- Медианное использование SM 40% (1 GPU может выполнять несколько задач).



SM - Streaming Multicore, TC - Tensor Core

Профайлинг

Модель: 123B InternLM с 2048 GPU A100

InternEvo-v1:

- 3D – Параллелизм

InternEvo-v2:

- Разделение параметров максимум до 64 GPU
- Перевычисление

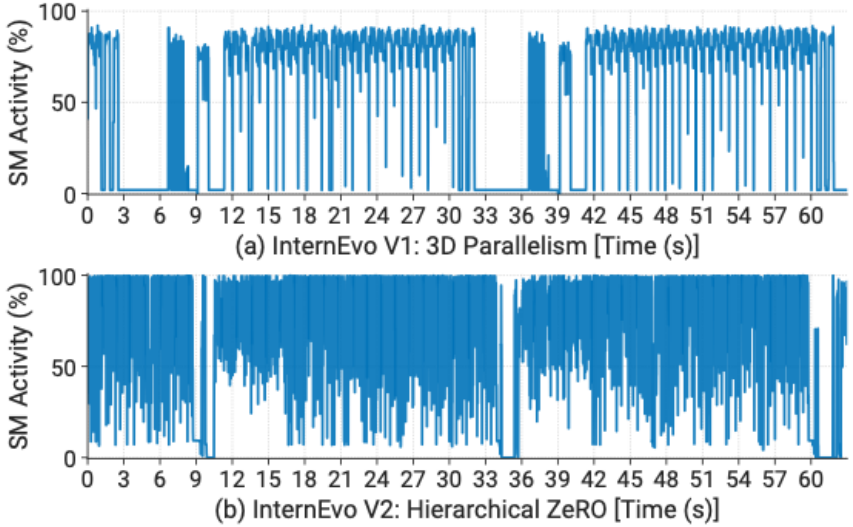
Использование SM в GPU

Основные улучшения в InternEvo-v2:

- Уменьшен простой GPU.
- Увеличено пиковое использование SM.

Под капотом:

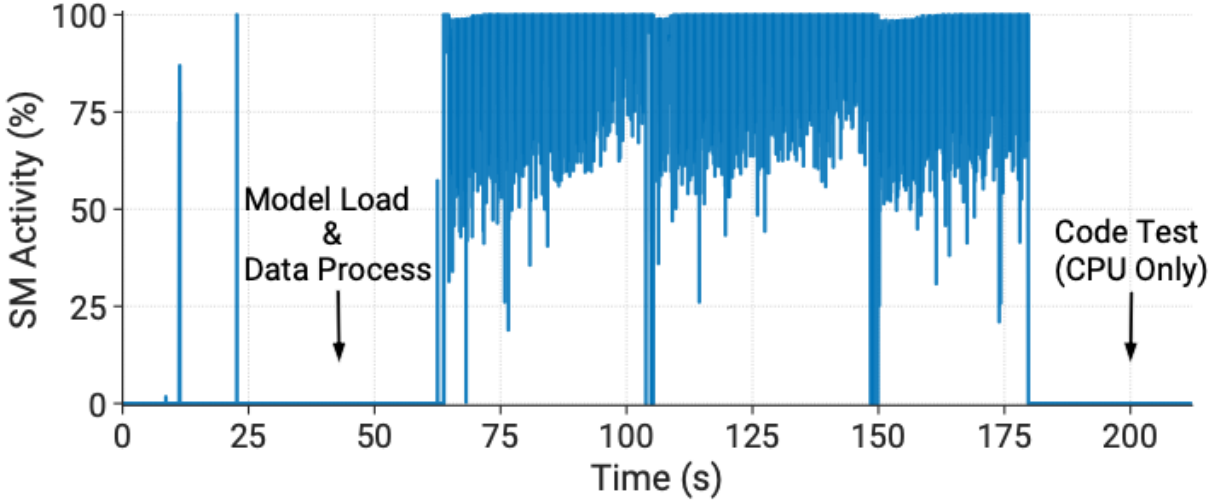
- Выборочное Communication Overlap (Технология NVidia).
- Стратегия гибридного параллелизма.



претренинг 123B InternLM на 2048 GPU A100

Использование SM в GPU, когда мало нагрузки

- Загрузка модели и обработка данных.
- Сбор метрик (До 30 минут).
- Пример: 7B HumanEval.
- Итог: Около 50% времени GPU не использовано.



Примечание: необходимость улучшения процесса расчета LLM на системном уровне

Failure Analysis Overview

- Analysis based on runtime logs and hardware monitor data from Kalos (32,500 tasks) and Seren (675 tasks)
- Focus on pretraining and inference tasks
- Goal: insights for fault-tolerance in LLMs

Failure Categories

- **Infrastructure:** NVLink, CUDA, Node, ECC, Network; high GPU usage and long recovery
- **Framework:** RuntimeError, ValueError, AttributeError; often config related
- **Script:** Programming/user errors; majority of failures

Key Observations

- Infrastructure failures consume >82% GPU resources, only 11% of job count
- High GPU temperatures in Kalos can cause NVLinkError/ECCError
- Auxiliary services (logging, monitoring) can trigger Connection/Network errors
- Evaluation jobs rarely fail ($\tilde{6}.7\%$)

Failure Statistics

Category	Reason	Num	GPU Demand		Time to Failure (mins)		GPU Time (mins)		Time to Restart (mins)			Cluster
			Average	Median	Average	Median	Average	Total%	Average	Median	TR/TF%	
Infrastructure	NVLink Error	54	800	896	868.1	155.3	585683	30.25%	95.6	0.2	11.02%	S, K
	CUDA Error	21	847	1024	923.2	586.0	785099	15.77%	78.3	2.0	8.48%	S, K
	Node Failure	16	712	768	1288.8	535.8	934394	14.30%	102.8	21.5	7.98%	S
	ECC Error	12	680	512	1303.4	1192.3	958404	11.00%	2.8	1.8	0.21%	S, K
	Network Error	12	758	768	549.6	310.1	394821	4.53%	592.1	7.4	107.74%	S, K
	Connection Error	147	29	1	51.9	0.5	24492	3.44%	0.8	0.0	1.51%	S, K
	S3 Storage Error	10	422	256	2317.8	202.2	222151	2.12%	6.2	0.2	0.27%	S
	NCCL Timeout Error	6	596	512	159.7	48.1	86856	0.50%	66.7	43.6	41.78%	K
	NCCL Remote Error	3	1152	1024	50.5	22.6	52419	0.15%	0.0	0.7	0.09%	K
Framework	Dataloader Killed	6	445	508	1580.6	961.4	764170	4.38%	115.1	0.9	7.28%	K
	Attribute Error	67	228	8	67.8	1.2	60914	3.90%	2.4	0.0	3.58%	S, K
	Out of Memory Error	14	572	640	323.8	14.5	245278	3.28%	122.7	1.2	37.89%	S, K
	Runtime Error	65	441	352	66.4	3.9	27667	1.72%	10.9	1.5	16.41%	S, K
	Assertion Error	105	413	256	41.7	3.0	12315	1.24%	185.9	1.6	445.87%	S, K
	Value Error	33	387	256	9.9	3.7	5049	0.16%	27.4	0.6	276.74%	S, K
	Zero Division Error	5	499	256	14.5	15.6	5363	0.03%	2.5	1.1	17.31%	S, K
	Model Loading Error	104	8	8	2.6	2.6	20	0.00%	0.0	0.0	0.00%	K
	Dataset Loading Error	5	1	1	1.6	1.6	1	0.00%	0.0	0.0	0.00%	K
Script	File Not Found Error	568	21	1	14.2	0.4	5210	2.83%	0.4	0.0	2.58%	S, K
	OS Error	266	8	1	9.6	0.8	1098	0.28%	0.3	0.0	3.17%	S, K
	Type Error	620	18	4	0.9	0.3	97	0.06%	0.2	0.0	28.27%	S, K
	Name Error	18	247	24	3.2	0.5	947	0.02%	2.9	2.4	90.92%	S, K
	Permission Error	7	438	512	4.3	0.8	2131	0.01%	2.4	2.2	56.38%	S
	Import Error	111	93	8	1.1	0.4	74	0.01%	0.7	0.0	63.68%	S, K
	Key Error	260	7	0	3.0	1.6	55	0.01%	0.1	0.0	2.10%	S, K
	Syntax Error	10	391	384	0.7	0.6	348	0.00%	1.7	1.7	261.73%	S, K
	Argument Error	3	344	512	0.7	0.7	288	0.00%	2.7	0.7	408.47%	S
	Called Process Error	4	256	256	0.2	0.2	52	0.00%	11.7	10.9	5714.29%	S
	Index Error	23	6	1	1.6	0.9	21	0.00%	0.8	0.0	49.73%	S, K

Failure Recovery

- Restart scenarios:
  1. Job error occurs
  2. Loss spike or abnormal training metrics
  3. Training process stuck
- Jobs revert to last checkpoint on restart
- Manual recovery needed due to lack of automatic support
- Framework improvements reduced losses in later models (123B)

Развернутые системы LLM

- Разработка LLM сталкивается с проблемами, но выявляет стратегии их преодоления.
- Два направления:
  1. Предобучение: повышение устойчивости к сбоям с помощью диагностики и автоматического восстановления.
  2. Оценка: быстрая реакция на задачи через декомпозицию.

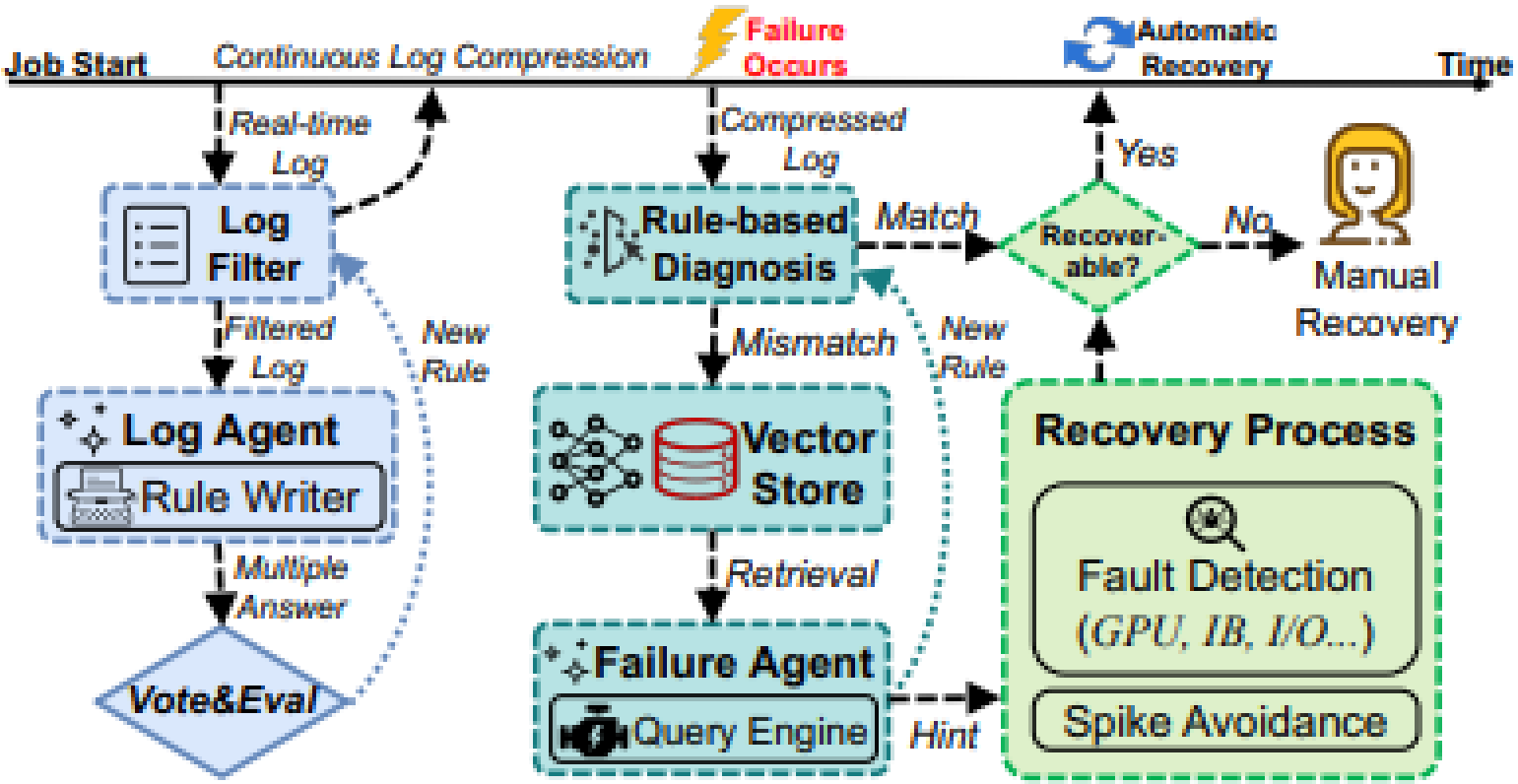
Устойчивое к сбоям предобучение

Мотивация: Предобучение LLM сопровождается сбоями из-за большого числа GPU и длительности обучения. Это снижает эффективность использования ресурсов и требует ручного вмешательства инженеров.

Система: Автоматическое выявление причин сбоев и восстановление включает три модуля:

1. **Чекпоинтинг:** частое сохранение модели с использованием асинхронного подхода для снижения накладных расходов.
2. **Диагностика:** комбинация правил и LLM для точного выявления причин сбоев.
3. **Восстановление:** обнаружение проблемных узлов и автоматический рестарт с последнего чекпоинта.

Диагностика сбоев и восстановление модели



- Рисунок демонстрирует поток обработки логов, фильтрацию и восстановление.
- Система значительно снижает ручное вмешательство (около 90%).

Разделённое планирование оценивания

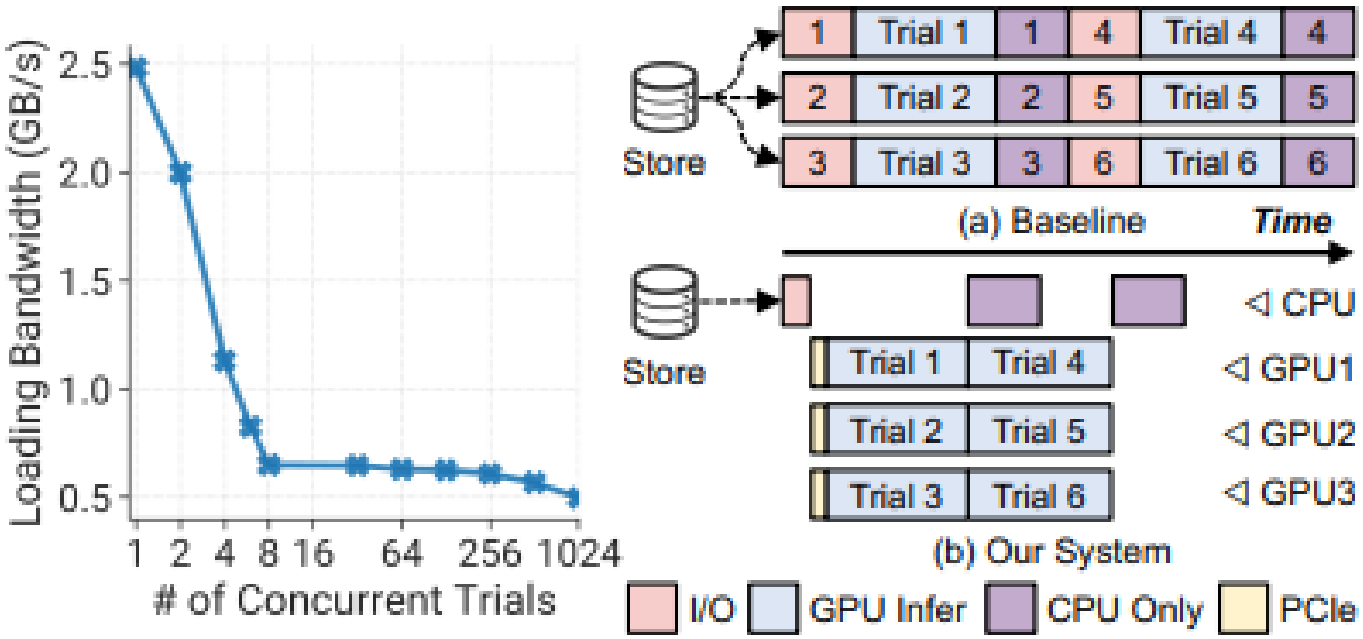
Мотивация: Оценка качества модели по одному метрику не даёт полной картины. Необходимо:

- Регулярная оценка на множестве задач для отслеживания прогресса.
- Быстрая обратная связь для корректировки обучения.

Система: Координатор задач реализует:

1. Разделение загрузки модели с удалённого хранилища и процесса оценивания.
2. Разделение вычисления метрик на отдельные CPU задачи.
3. Приоритетное планирование на основе ожидаемой длительности задач.

Нагрузочное тестирование и планирование оценивания



- Левая часть: При 8 задачах загрузка падает из-за NIC — узкое место.
- Правая часть: Модель в CPU-кэш, оценка через PCIe — без повторной загрузки, GPU не ждёт.
- Декомпозиция повышает эффективность и сокращает время простоя GPU.

Масштаб исследования

- Дата-центр Acme: 4 704 GPU A100
- 1.09 млн LLM-задач за 6 месяцев
- Среднее число GPU на задачу - 6.3
- Модели от 7B до 123B параметров
- Анализ этапа обучения (serving вынесен в отдельный кластер)

Важно: исследование основано на промышленной эксплуатации, а не на лабораторном эксперименте.

Выявленные системные проблемы

- Падение throughput в 2–3 раза из-за Garbage Collection
- До 30% времени шага занимала функция `list_traverse`
- Утечка памяти в PyTorch Dataloader -> сбой в среднем через 27 часов
- Низкая утилизация GPU в MoE на 1024 GPU
- Узкое место - all-to-all сетевые коммуникации

Вывод: LLM-нагрузки создают новые инфраструктурные риски.

Ключевые выводы статьи

- LLM-нагрузки отличаются от классического DL
- Сеть и межузловые коммуникации становятся критическим фактором
- Отказы оказывают сильное влияние при длительном pretraining
- Существующие кластеры не оптимизированы под масштаб LLM

Главный результат: обучение LLM требует специализированной архитектуры и системной оптимизации.

Направления развития

- Оптимизация long-sequence pretraining
- Улучшение поддержки MoE
- Повышение эффективности RLHF
- Модернизация сетевой инфраструктуры (NIC)
- Расширение кластера для более масштабных моделей

Итог: статья формирует инженерные принципы построения дата-центров для LLM.